```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix , ConfusionMatrixDisplay,
classification_report

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam

tf.random.set_seed(42)
np.random.seed(42)

train=pd.read_csv("fashion-mnist_train.csv")
test=pd.read_csv("fashion-mnist_test.csv")

train.head()

{"type":"dataframe","variable_name":"train"}

y_train=train.iloc[:,0].values.astype("int64")
x_train=train.iloc[:,1:].values.astype("float32").reshape(-1,28,28)

print(x_train.shape)
print(y_train.shape)

(60000, 28, 28)
(60000,)

y_test=test.iloc[:,0].values.astype("int64")
x_test=test.iloc[:,1:].values.astype("float32").reshape(-1,28,28)

print(x_test.shape)
print(y_test.shape)

print(np.unique(y_train))

(10000, 28, 28)
(10000,)
[0 1 2 3 4 5 6 7 8 9]

x_train=x_train/255.0
x_test=x_test/255.0

x_train=x_train.reshape(-1,28,28,1)
x_test=x_test.reshape(-1,28,28,1)

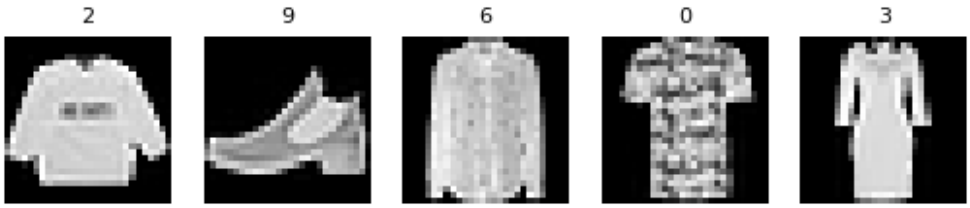
y_train_sc=to_categorical(y_train,10)
```

```
y_test_sc=to_categorical(y_test,10)

print(x_train.min(), x_train.max())
```

```
0.0 1.0
```

```
for i in range(5):
  plt.subplot(1,5,i+1)
  plt.imshow(x_train[i].reshape(28,28), cmap="gray")
  plt.title(y_train[i], fontsize=8)
  plt.axis("off")
plt.show()
```




```
model=keras.Sequential([
    layers.Input(shape=(28,28,1)),
    layers.Conv2D(32,(3,3), activation="relu"),
    layers.MaxPooling2D((2,2,)),
    layers.Conv2D(64,(3,3),activation="relu"),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(128,activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(10,activation="softmax")
])
model.summary()
```

```
Model: "sequential_1"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
┃ conv2d_4 (Conv2D)                    ┃ (None, 26, 26, 32)          ┃
320 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━┫
┃ max_pooling2d_4 (MaxPooling2D)       ┃ (None, 13, 13, 32)          ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━┫
```

```
| conv2d_5 (Conv2D)                    | (None, 11, 11, 64)      |
18,496 |
├──────────────────────────────────────┼─────────────────────────┼──────────
──────┤
| max_pooling2d_5 (MaxPooling2D)       | (None, 5, 5, 64)        |
0 |
├──────────────────────────────────────┼─────────────────────────┼──────────
──────┤
| flatten_2 (Flatten)                  | (None, 1600)            |
0 |
├──────────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dense_3 (Dense)                      | (None, 128)             |
204,928 |
├──────────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dropout_2 (Dropout)                  | (None, 128)             |
0 |
├──────────────────────────────────────┼─────────────────────────┼──────────
──────┤
| dense_4 (Dense)                      | (None, 10)              |
1,290 |
└──────────────────────────────────────┴─────────────────────────┴──────────
──────┘

 Total params: 225,034 (879.04 KB)

 Trainable params: 225,034 (879.04 KB)

 Non-trainable params: 0 (0.00 B)
```

```
model.compile(
    loss="categorical_crossentropy",
    optimizer=Adam(learning_rate=0.001),
    metrics=["accuracy"]
)
history=model.fit(
    x_train,
    y_train_sc,
    validation_split=0.1,
    epochs=20,
    batch_size=128,
    verbose=2
)
```

```
Epoch 1/20
422/422 - 7s - 17ms/step - accuracy: 0.7770 - loss: 0.6192 -
val_accuracy: 0.8528 - val_loss: 0.4118
Epoch 2/20
422/422 - 2s - 5ms/step - accuracy: 0.8554 - loss: 0.3978 -
```

```
val_accuracy: 0.8748 - val_loss: 0.3416
Epoch 3/20
422/422 - 2s - 4ms/step - accuracy: 0.8754 - loss: 0.3420 -
val_accuracy: 0.8852 - val_loss: 0.3207
Epoch 4/20
422/422 - 2s - 4ms/step - accuracy: 0.8874 - loss: 0.3137 -
val_accuracy: 0.8880 - val_loss: 0.3054
Epoch 5/20
422/422 - 2s - 4ms/step - accuracy: 0.8937 - loss: 0.2893 -
val_accuracy: 0.9008 - val_loss: 0.2835
Epoch 6/20
422/422 - 2s - 4ms/step - accuracy: 0.9003 - loss: 0.2723 -
val_accuracy: 0.9023 - val_loss: 0.2802
Epoch 7/20
422/422 - 2s - 4ms/step - accuracy: 0.9064 - loss: 0.2540 -
val_accuracy: 0.9038 - val_loss: 0.2769
Epoch 8/20
422/422 - 2s - 4ms/step - accuracy: 0.9124 - loss: 0.2375 -
val_accuracy: 0.9053 - val_loss: 0.2702
Epoch 9/20
422/422 - 2s - 4ms/step - accuracy: 0.9173 - loss: 0.2269 -
val_accuracy: 0.9095 - val_loss: 0.2672
Epoch 10/20
422/422 - 2s - 4ms/step - accuracy: 0.9215 - loss: 0.2133 -
val_accuracy: 0.9092 - val_loss: 0.2623
Epoch 11/20
422/422 - 2s - 4ms/step - accuracy: 0.9262 - loss: 0.2011 -
val_accuracy: 0.9133 - val_loss: 0.2591
Epoch 12/20
422/422 - 2s - 4ms/step - accuracy: 0.9295 - loss: 0.1889 -
val_accuracy: 0.9132 - val_loss: 0.2590
Epoch 13/20
422/422 - 2s - 4ms/step - accuracy: 0.9340 - loss: 0.1782 -
val_accuracy: 0.9120 - val_loss: 0.2629
Epoch 14/20
422/422 - 2s - 5ms/step - accuracy: 0.9373 - loss: 0.1702 -
val_accuracy: 0.9102 - val_loss: 0.2631
Epoch 15/20
422/422 - 2s - 4ms/step - accuracy: 0.9396 - loss: 0.1591 -
val_accuracy: 0.9113 - val_loss: 0.2720
Epoch 16/20
422/422 - 2s - 4ms/step - accuracy: 0.9436 - loss: 0.1507 -
val_accuracy: 0.9125 - val_loss: 0.2792
Epoch 17/20
422/422 - 2s - 4ms/step - accuracy: 0.9459 - loss: 0.1428 -
val_accuracy: 0.9172 - val_loss: 0.2698
Epoch 18/20
422/422 - 2s - 4ms/step - accuracy: 0.9501 - loss: 0.1326 -
val_accuracy: 0.9180 - val_loss: 0.2611
```

```
Epoch 19/20
422/422 - 2s - 4ms/step - accuracy: 0.9505 - loss: 0.1312 -
val_accuracy: 0.9155 - val_loss: 0.2794
Epoch 20/20
422/422 - 2s - 4ms/step - accuracy: 0.9538 - loss: 0.1241 -
val_accuracy: 0.9173 - val_loss: 0.2790
```

```
test_loss, test_acc=model.evaluate(x_test,y_test_sc)
print(test_loss)
print(test_acc)
```

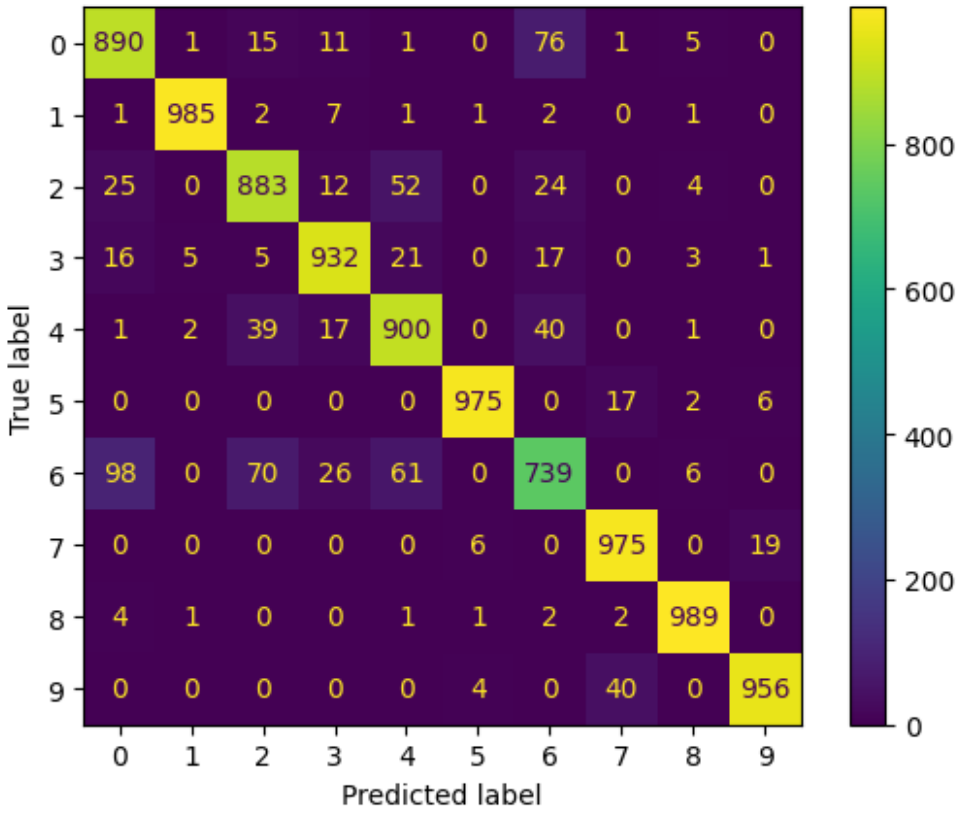
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9224 - loss:
0.2528
0.2527722120285034
0.9223999977111816
```

```
y_pred=model.predict(x_test).argmax(axis=1)
print(classification_report(y_test,y_pred))
ConfusionMatrixDisplay.from_predictions(y_test,y_pred)
plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step
              precision    recall  f1-score   support

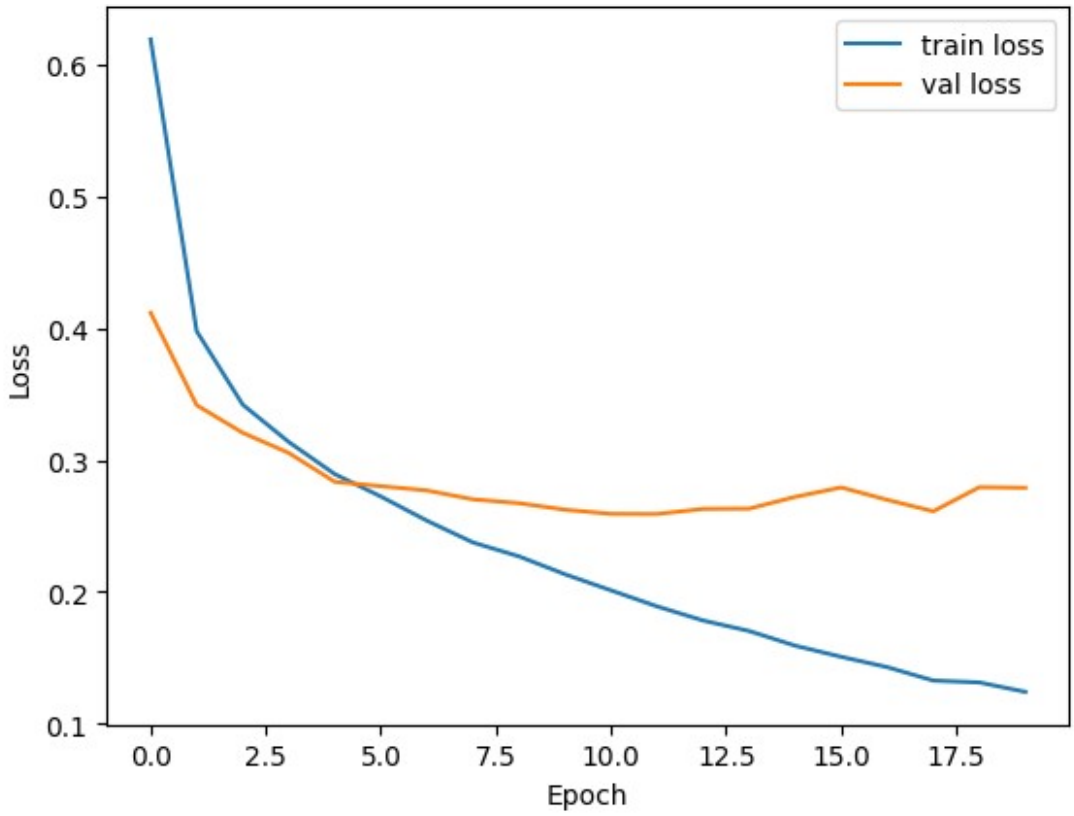
           0       0.86      0.89      0.87      1000
           1       0.99      0.98      0.99      1000
           2       0.87      0.88      0.88      1000
           3       0.93      0.93      0.93      1000
           4       0.87      0.90      0.88      1000
           5       0.99      0.97      0.98      1000
           6       0.82      0.74      0.78      1000
           7       0.94      0.97      0.96      1000
           8       0.98      0.99      0.98      1000
           9       0.97      0.96      0.96      1000

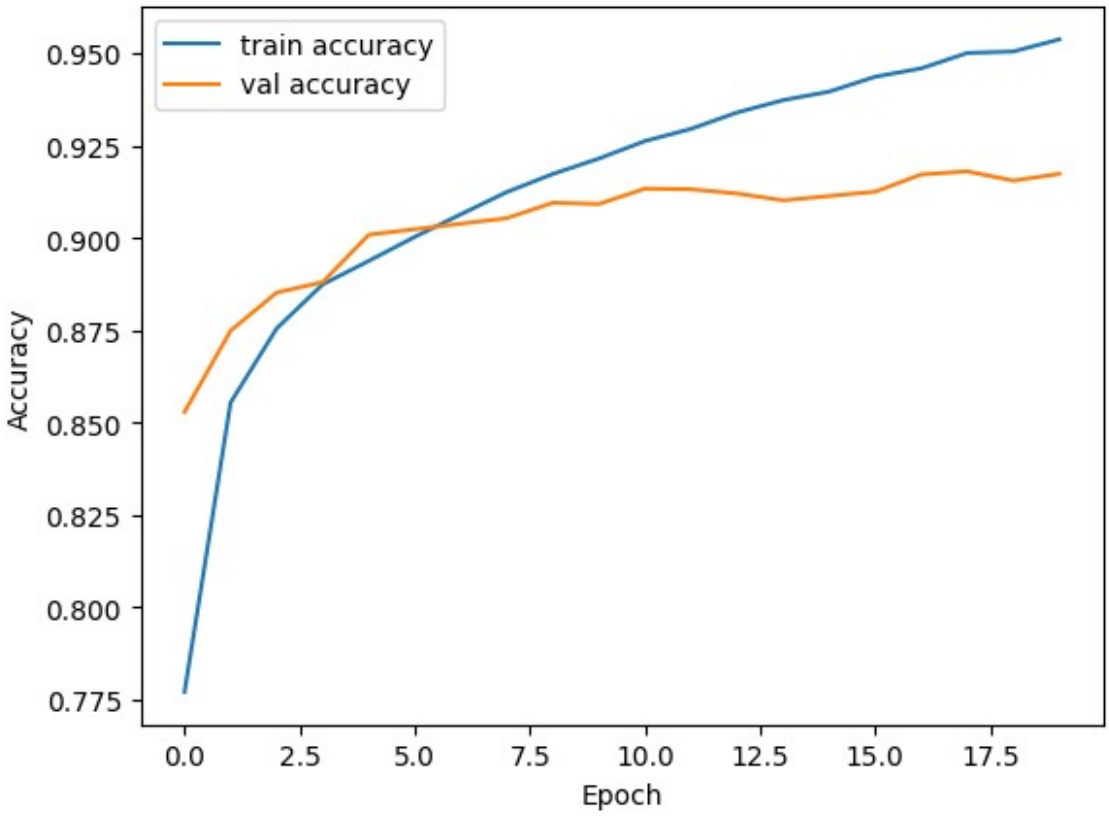
    accuracy                           0.92     10000
   macro avg       0.92      0.92      0.92     10000
weighted avg       0.92      0.92      0.92     10000
```




```
plt.plot(history.history["loss"], label="train loss")
plt.plot(history.history["val_loss"], label="val loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

plt.plot(history.history["accuracy"], label="train accuracy")
plt.plot(history.history["val_accuracy"], label="val accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```


train loss
val loss
Loss
Epoch
0.6
0.5
0.4
0.3
0.2
0.1
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5


train accuracy
val accuracy
Accuracy
Epoch
0.950
0.925
0.900
0.875
0.850
0.825
0.800
0.775
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5